RAPPORT DE PROJET · 2D AND 3D PRINTING

Sketch2CAD-Local

Du croquis papier coté au plan 2D et au solide 3D,
par une IA neuro-symbolique exécutée 100 % en local

Architecture · algorithmes · interface web · résultats · déploiement

Version du code : 0.1 (pipeline v1.2) · 28 octobre 2026

Résumé. Sketch2CAD-Local transforme la photographie d'un croquis mécanique dessiné à main levée et coté à la main en fichiers exploitables par une machine : mise en plan 2D cotée (DXF, SVG) et solide 3D (STEP, STL), accompagnés d'un script CAO Python paramétrique. Le croquis n'a pas besoin d'être à l'échelle : les cotes manuscrites font foi et un solveur de contraintes recalcule la géométrie exacte. Tout s'exécute sur le poste de l'utilisateur, sans envoi de données vers un service externe.

Indicateur	Valeur mesurée
Formes reconnues	pièce plate extrudée (contour, congés, perçages), cylindre, corps de révolution (gélule, cône, tronc, hémisphère), cercle (sphère ou disque au choix), dessin ouvert signalé
Précision géométrique	cotes satisfaites à moins de 10^{-13} mm sur les cas de test (tolérance de contrôle : 0,01 mm)
Croquis réels testés	3 photos : cylindre (Ø 1000 × 2000 mm), feuille de 6 dessins, cercle
Tests automatiques	34 tests pytest, tous réussis
Taille du code	≈ 4 800 lignes (Python, page web, tests)
Exécution	100 % locale, CPU suffisant ; GPU optionnel via Ollama (modèles Qwen)

Sommaire

Sommaire 2

1. Contexte et problématique 3

2. Objectifs et contraintes 3

3. Architecture générale 4

3.1 Les agents . . . 4

3.2 Flux de données . . . 5

4. Agent 1 — Perception 5

4.1 Prétraitement de l'image . . . 5

4.2 Lecture des cotes (OCR) et analyse . . . 5

4.3 Découpage d'une feuille en dessins . . . 6

4.4 Contour et perçages (pièce plate) . . . 6

5. Reconnaissance des formes 8

5.1 Cylindre . . . 8

5.2 Corps de révolution . . . 8

6. Agent 2 — Graphe de contraintes 10

7. Agent 3 — Solveur de contraintes 10

8. Agent 5 — Critique et auto-correction 11

9. Agent 4 — Synthèse « Code-as-CAD » 12

10. Interface web de test 13

11. Résultats et validation 14

11.1 Croquis traités . . . 14

11.2 Tests automatiques (34, tous réussis) . . . 15

11.3 Temps de calcul (environnement de test, CPU) . . . 15

12. Matériel, IA locale et utilisation du GPU 15

13. Installation et utilisation 16

14. Organisation du code 16

15. Limites et perspectives 17

Limites actuelles . . . 17

Feuille de route . . . 17

Conclusion . . . 17

1. Contexte et problématique

En ingénierie mécanique, en prototypage rapide et en atelier, la conception d'une pièce commence presque toujours par un croquis à main levée sur une feuille de papier. Le transformer en fichier exploitable par une machine à commande numérique (CNC, découpe laser, impression 3D) impose de redessiner la pièce dans un logiciel de CAO (SolidWorks, AutoCAD, FreeCAD, Catia) : une tâche répétitive et chronophage.

Les approches d'IA purement générative (diffusion d'images, génération de maillages à partir d'une image) échouent dans ce contexte industriel pour trois raisons :

- **Absence d'échelle.** Un segment dessiné deux fois plus court qu'un autre peut porter une cote deux fois plus grande (150 mm contre 80 mm) : la géométrie dessinée ne vaut rien, seules les cotes comptent.
- **Contraintes numériques exactes.** Un réseau génératif produit une forme plausible mais approximative, inutilisable en usinage.
- **Confidentialité.** Les bureaux d'études refusent d'envoyer les plans de pièces propriétaires vers des API externes.

2. Objectifs et contraintes

Exigence	Réponse apportée
Cotes manuscrites respectées exactement	solveur algébrique de contraintes (moindres carrés), contrôle à 0,01 mm, rapport des conflits et des degrés de liberté restants
Croquis hors échelle	la géométrie dessinée ne sert qu'à la topologie et à l'estimation initiale
Sorties 2D et 3D	DXF et SVG cotés (ezdxf), STEP et STL (noyau OpenCASCADE via build123d), script part.py
100 % local	aucun appel réseau pendant le traitement ; interface servie sur 127.0.0.1 ; visionneuse 3D embarquée
Matériel modeste (GPU < 12 Go ou CPU)	chaque étape neuronale a un repli déterministe ; le profil offline fonctionne sans aucun modèle de langage
Résultat vérifiable	statut honnête (OK / OK avec avertissements / À VÉRIFIER / CHOIX REQUIS), images de contrôle, fichiers intermédiaires JSON rejouables

3. Architecture générale

Le système est **neuro-symbolique** : la perception (lecture de l'image et des cotes) est séparée de la résolution mathématique et de la génération CAO. Aucun réseau neuronal ne produit de coordonnée : il ne fait, au plus, que lire ou rattacher des cotes. Les coordonnées sortent d'un solveur déterministe, et les solides d'un noyau CAO industriel.

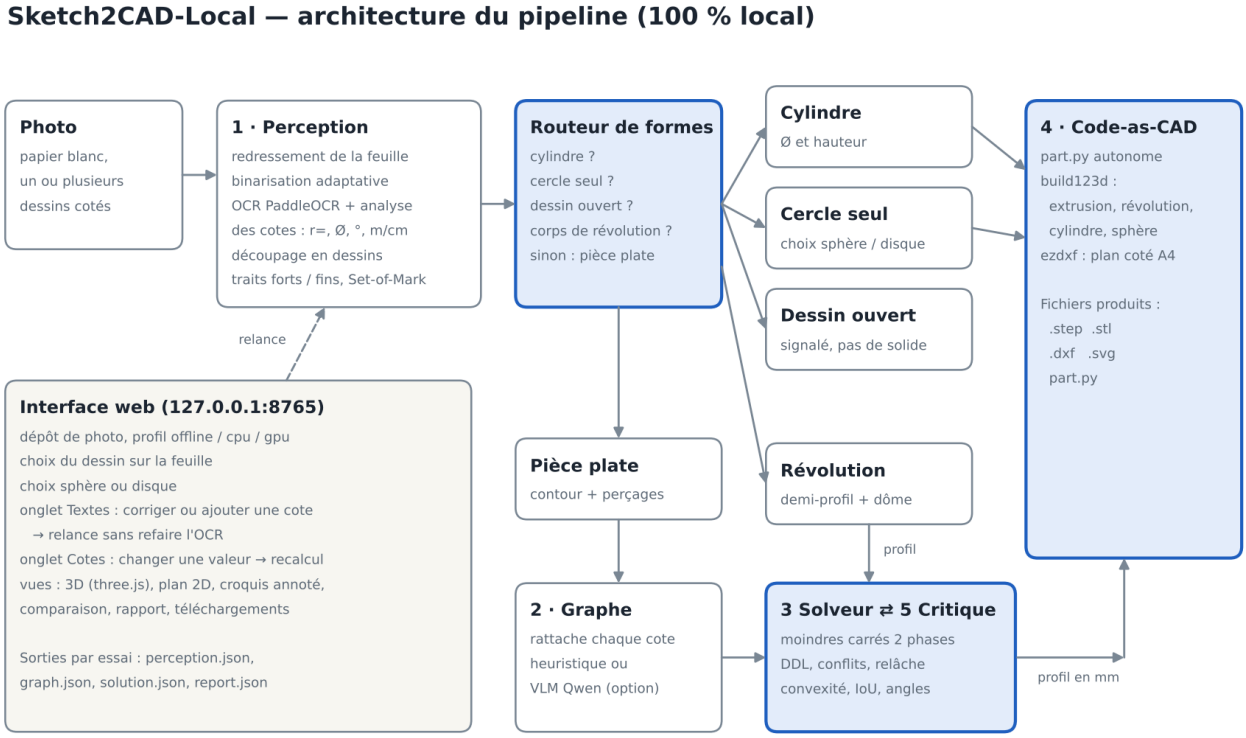


Figure 1 — Architecture du pipeline : perception, routeur de formes, branches spécialisées, solveur et critique, génération CAO, interface web.

3.1 Les agents

Agent	Rôle	Technique	Module
1 · Perception	image propre, dessins, contour, perçages, textes	OpenCV, PaddleOCR, Set-of-Mark	perception.py, ocr.py, sheet.py
Routeur de formes	décide de la nature du dessin	tests géométriques successifs	primitives.py, revolution.py, perception.py
2 · Graphe	rattache chaque cote à l'élément qu'elle mesure	affectation optimale (hongroise) ou VLM	graph_builder.py, llm.py
3 · Solveur	coordonnées exactes en mm	moindres carrés non linéaires (SciPy)	solver.py
5 · Critique	détecte et corrige les erreurs d'interprétation	convexité, IoU, rapports cote/dessin	critic.py
4 · Code-as-CAD	écrit et exécute part.py	build123d, ezdxf	cad.py, profile.py
Orchestration	enchaîne les agents, statut, rapport	—	pipeline.py, cli.py, webapp.py

3.2 Flux de données

Chaque essai écrit ses étapes dans son dossier : `texts_all.json` (textes OCR bruts), `perception.json` (dessins, forme reconnue, primitives en pixels), `graph.json` (graphe de contraintes), `solution.json` (géométrie en mm), `report.json` (statut, avertissements, conflits), les images `sheet.png`, `som.png`, `comparison.png`, puis les livrables `part.py / .step / .stl / .dxf / .svg`. Toute étape peut être rejouée à partir du fichier précédent : par exemple, un `graph.json` corrigé à la main relance seulement le solveur et la CAO.

4. Agent 1 — Perception

4.1 Prétraitement de l'image

- **Redressement de la feuille** : recherche du plus grand quadrilatère clair (30 à 98 % de l'image), homographie vers un rectangle.
- **Normalisation de l'éclairage** : division de l'image par un flou gaussien large (σ = taille/30), qui efface ombres et dégradés.
- **Binarisation adaptative** (bloc 31 px) et ouverture morphologique contre le grain du papier.
- **Épaisseur des traits** : transformée de distance mesurée sur les crêtes des traits ; un seuil d'Otsu sépare traits forts (contour, ISO 128) et traits fins (cotation) si les deux populations diffèrent d'au moins 30 %. Sans cette séparation, les lignes de cote se soudent au contour.

4.2 Lecture des cotes (OCR) et analyse

PaddleOCR 3.x (modèles PP-OCRv6, sur CPU, oneDNN désactivé pour contourner une erreur de certaines versions) lit les textes ; Tesseract et EasyOCR servent de replis. Les jetons voisins d'une même ligne sont fusionnés (« ep » + « 5 »). Chaque texte est ensuite typé par expressions régulières :

Écriture sur le croquis	Type	Exemple → valeur
nombre seul	longueur	150 → 150 mm
suffixe °	angle	120° → 120°
préfixe R	rayon de congé	R10 → 10 mm
préfixe Ø, ⌀, D	diamètre	Ø12 → 12 mm
ep, e=, t=	épaisseur	ep 5 → 5 mm
r= (r manuscrit souvent lu F= ou Γ=)	rayon	F=0.5 → 0,5
d=	diamètre ou longueur (tranché par la position)	d=2m → 2000 mm
h=, l=, L=	longueur / hauteur	h=40cm → 400 mm
unités m, cm, mm	conversion en mm	une cote sans unité prend l'unité écrite sur les autres
« 012 »	Ø lu comme 0 : ambigu	tranché par la proximité d'un perçage

Les textes illisibles (« r= 0r », « u B ») ne sont plus jetés : ils sont conservés avec le type *illisible*, ce qui fait passer le résultat en « À VÉRIFIER » et permet de les corriger dans l'interface. Le masquage des textes retire uniquement les traits situés à plus de 70 % dans une boîte de texte : un trait de contour qui passe près d'une cote n'est plus coupé.

4.3 Découpage d'une feuille en dessins

Les traits (poussières retirées) sont dilatés de 2,5 % de la taille de l'image ; chaque composante assez grande devient un dessin. Les rectangles qui se chevauchent sont fusionnés (un trait fragmenté ou un perçage restent dans leur dessin), les composantes faites uniquement de texte sont écartées, et chaque texte est rattaché au dessin le plus proche. Un dessin touchant le bord de la photo est marqué *coupé* : il n'entraîne pas de question à l'utilisateur.

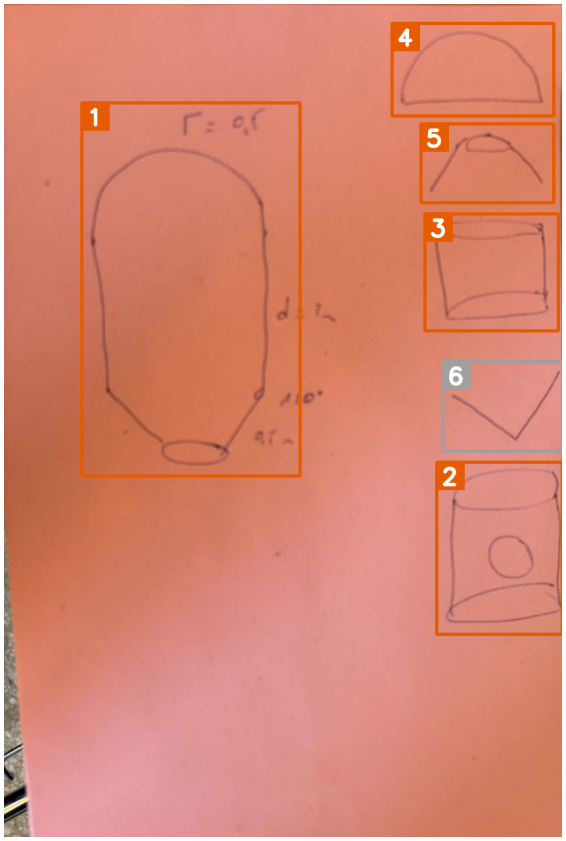


Figure 2 — Votre feuille : 6 dessins détectés et numérotés (le n° 6, coupé par le bord, en gris).

4.4 Contour et perçages (pièce plate)

- contour : fermeture morphologique, remplissage du plus grand contour extérieur, ouverture (noyau 4 × épaisseur) pour retirer les lignes d'attache collées ;
- polygone : `approxPolyDP` (1,2 % du périmètre), fusion des sommets distants de moins de 3 % (congés dessinés) en coin vif, suppression des sommets plats (> 170°) ;
- perçages : régions intérieures quasi circulaires (circularité > 0,75), cherchées sur les traits non masqués ;
- Set-of-Mark : image annotée V* (sommets), E* (arêtes), C* (perçages), T* (textes), pour l'utilisateur et le VLM.

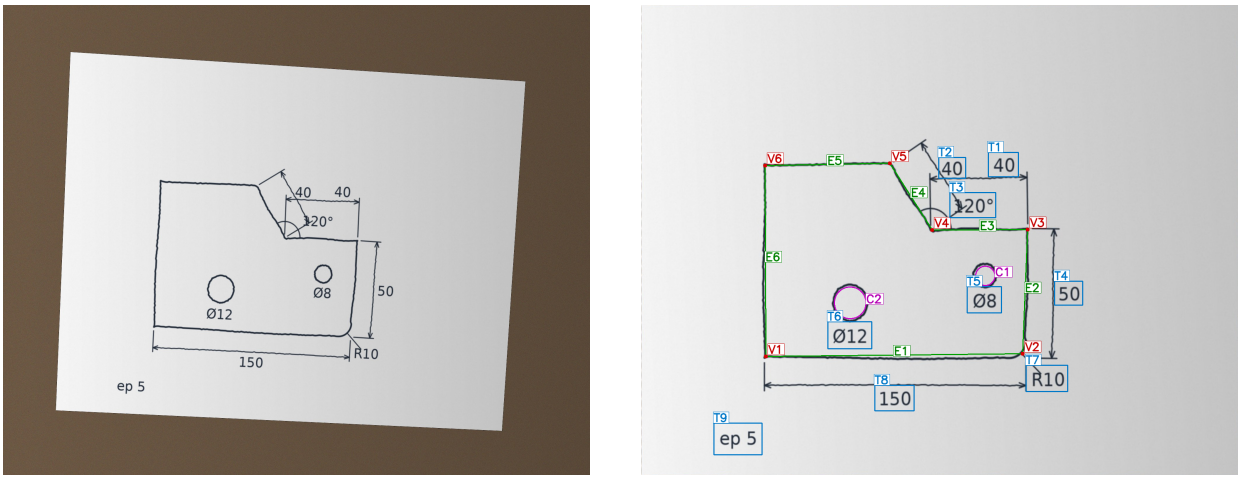


Figure 3 — Croquis de test photographié en biais (volontairement hors échelle) et image Set-of-Mark produite par l'agent 1.

5. Reconnaissance des formes

Après la perception, un routeur teste les formes dans un ordre fixe ; la première qui répond oriente le traitement. Chaque test exige des indices géométriques précis, pour éviter qu'une pièce plate soit prise pour un solide de révolution (des tests de non-régression le vérifient).

Ordre	Forme	Critères principaux	Traitement
1	Cylindre en perspective	2 génératrices quasi verticales (Hough, ±14°), parallèles, écart ≥ 25 % de la longueur ; ellipse ajustée à chaque base (≥ 50 % des points sur l'ellipse, grand axe 0,75–1,35 × Ø) ; bases distinctes	Ø et hauteur depuis les cotes
2	Cercle seul	circularité ≥ 0,82 et rapport des axes ≥ 0,85	question : sphère ou disque
3	Dessin ouvert	contour fermé < 20 % du rectangle englobant	signalé, aucun solide
4	Corps de révolution	silhouette symétrique (IoU miroir ≥ 0,82) + indice 3D : ellipse de base ou dôme	demi-profil → solveur → révolution
5	Pièce plate	par défaut	contour → graphe → solveur → extrusion

5.1 Cylindre

Le trait d'un stylo pâle sur papier froissé est très fragmenté (trous de 30 à 80 px) : fermer le contour par morphologie échoue. Le cylindre est donc détecté par géométrie : segments de Hough regroupés en génératrices, puis ajustement d'ellipse sur les traits situés autour de chaque extrémité. Les cotes sont rattachées par position : une cote écrite à côté des génératrices est la hauteur, une cote dans une base est le rayon ou le diamètre. Si une seule dimension est cotée, l'autre est déduite des proportions du dessin, avec un avertissement.

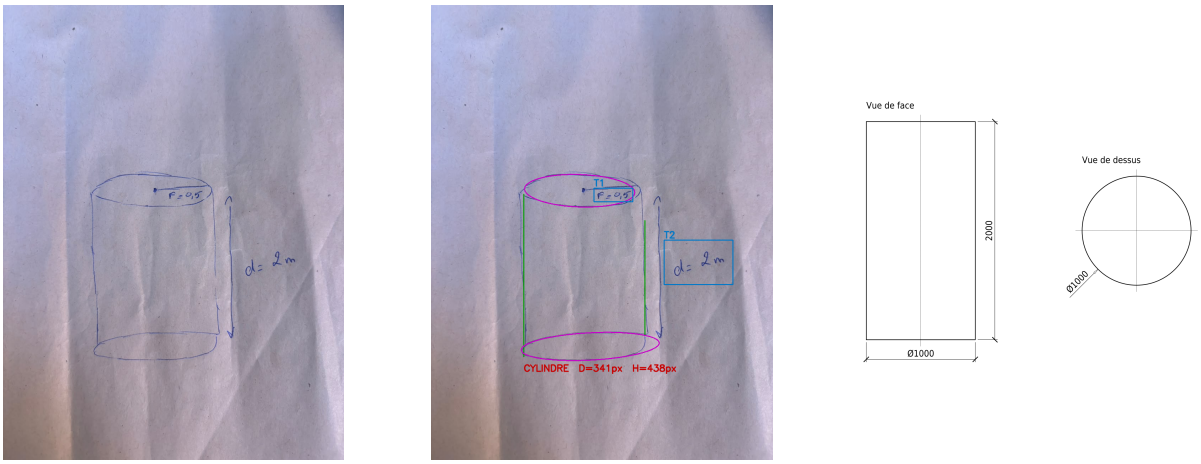


Figure 4 — Votre photo de cylindre (« r=0,5 » lu « F=0.5 », « d=2m »), détection (génératrices en vert, bases en violet) et plan obtenu : Ø 1000 × 2000 mm.

5.2 Corps de révolution

- **Silhouette** : fermeture morphologique ; si le contour n'est pas fermé (coin non rejoint, base non tracée), enveloppe convexe des traits.
- **Axe** : médiane des milieux de chaque ligne de la silhouette ; symétrie mesurée par l'IoU entre la silhouette et son miroir.
- **Bases elliptiques** : régions intérieures horizontales dont l'IoU avec l'ellipse ajustée dépasse 0,85 (une bande rectangulaire entre deux traits est rejetée) ; la face plane est au niveau du centre de l'ellipse.

- **Dôme** : le profil part de l'axe et s'élargit jusqu'à ce que le flanc devienne vertical (|dr/dy| < 0,15) ; cercle ajusté avec centre sur l'axe, accepté si l'erreur moyenne est < 5 % du rayon.
- **Demi-profil** : polygone axe-bas → bord de base → flanc → fin du dôme → sommet sur l'axe, simplifié (sommets alignés à 15° près et segments trop courts supprimés).
- **Base dessinée de biais** (photo inclinée) : la face est placée là où le profil atteint sa pleine largeur.

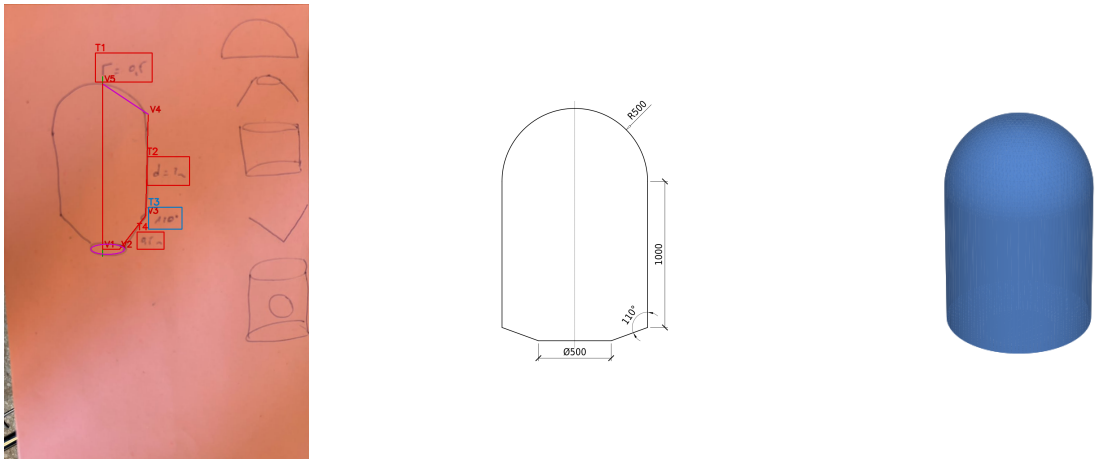


Figure 5 — Dessin n° 1 de votre feuille : demi-profil détecté (textes illisibles encadrés en rouge), plan et solide après correction des textes : dôme R 500, partie droite 1000 mm, base Ø 500 mm.

6. Agent 2 — Graphe de contraintes

L'agent 2 rattache chaque cote lue à l'élément qu'elle mesure et produit `graph.json`. Il ne calcule aucune coordonnée.

- **Heuristique déterministe** : pour chaque type de cote, affectation optimale (algorithme hongrois) entre textes et éléments selon la distance texte–élément (milieu d'arête pour une longueur, sommet pour un angle ou un rayon, cercle pour un diamètre) ; une cote trop éloignée reste non rattachée.
- **Contraintes implicites** : une arête à moins de 6° de l'horizontale ou de la verticale reçoit une contrainte implicite, relâchable si elle contredit une cote écrite.
- **VLM (option)** : un modèle vision-langage local reçoit l'image Set-of-Mark et la liste des textes, et répond en JSON contraint par un schéma ; une réponse invalide est relancée avec le message d'erreur, et chaque rattachement est validé (type compatible, élément existant) avant usage. Les rattachements manquants sont complétés par l'heuristique.

Pour un corps de révolution, le graphe est construit sur le demi-profil : axe vertical fixe, faces planes horizontales, dôme, et cotes rattachées après avoir ramené par symétrie les textes écrits à gauche de l'axe.

```
{ "vertices": [{"id": "V1", "px": [274, 684]}, ...],
  "edges":    [{"id": "E1", "from": "V1", "to": "V2"}, ...],
  "constraints": [
    {"type": "vertical",    "edge": "E5", "note": "axe de révolution"},
    {"type": "arc_radius",  "edge": "E4", "value": 500.0,  "source": "T1"},
    {"type": "length",      "edge": "E3", "value": 1000.0, "source": "T2"},
    {"type": "angle",       "vertex": "V3", "value": 110.0, "source": "T3"},
    {"type": "length",      "edge": "E1", "value": 250.0,  "source": "T4"},   # rayon de base = Ø/2
    {"type": "arc_tangent", "edge": "E4", "implicit": true}  ] }
```

Extrait de graph.json pour la gélule de votre feuille.

7. Agent 3 — Solveur de contraintes

Les inconnues sont les coordonnées (x, y) de chaque sommet (en mm, y vers le haut) ; le premier sommet est fixé à l'origine. Chaque contrainte devient un résidu, et le problème est résolu par moindres carrés non linéaires (`scipy.optimize.least_squares`, région de confiance).

Contrainte	Résidu (nul quand la contrainte est satisfaite)
longueur L de l'arête ab	$\lVert P_b - P_a \rVert - L$
angle intérieur θ au sommet k	$\text{angle}(P_{k+1} - P_k,\ P_{k-1} - P_k) - \theta$, ramené dans $[-\pi, \pi]$ et multiplié par une longueur de référence
horizontale / verticale	Δy / Δx de l'arête (poids 0,1 si implicite)
perpendiculaire / parallèle	produit scalaire / vectoriel normalisé
distance d'un perçage à une arête	distance signée − d
rayon ρ du dôme (arête D → E, E sur l'axe)	$\sqrt{((x_D - x_E)^2 + (y_D - y_E + \rho)^2)} - \rho$
dôme hémisphérique tangent (implicite)	$(y_E - y_D) - (x_D - x_E)$
rayon r à un sommet (révolution)	$x_k - x_{axe} - r$

Stratégie de résolution

- **Estimation initiale** : échelle mm/px = médiane des rapports cote / longueur dessinée ; les pixels sont multipliés par cette échelle.

- **Phase 1** : résolution avec une faible régularisation vers la forme dessinée (10^{-3}), qui lève l'indétermination des croquis sous-cotés.
- **Phase 2** : résolution sans régularisation à partir de la phase 1, pour satisfaire exactement les cotes.
- **Relâche** : si une cote écrite n'est pas satisfaite alors qu'une contrainte implicite l'est mal, la contrainte implicite la plus violée est retirée et le calcul relancé (les cotes écrites priment toujours).
- **Diagnostic** : degrés de liberté restants = nombre d'inconnues − rang du jacobien (SVD), rotation libre déduite ; toute cote non satisfaite à 0,01 mm est listée comme conflit.

Les congés et diamètres de perçage ne passent pas par le solveur : ils sont appliqués sur le profil résolu, avec contrôle que chaque congé tient dans ses deux arêtes.

8. Agent 5 — Critique et auto-correction

L'agent critique compare le profil résolu au croquis et renvoie le travail au solveur s'il détecte une erreur d'interprétation ; la boucle s'arrête après 3 itérations.

Contrôle	Ce qu'il détecte	Action
Signature de convexité	un angle pris dans le mauvais sens (120° écrit pour un coin rentrant de 240°)	θ devient 360° − θ, nouvelle résolution
Rapport cote / dessin	une cote 3 fois plus grande ou plus petite que la médiane	rattachement à une arête plus cohérente, sinon lecture OCR signalée
Ressemblance (IoU après alignement affine)	forme résolue très différente du dessin (IoU < 0,6)	signalé
Auto-intersection, sens	profil qui se recoupe, miroir	signalé
Perçages	perçage hors matière ou paroi < 1 mm	signalé
Angle de révolution	angle écrit à plus de 30° de l'angle dessiné	avertissement : sens de mesure à vérifier
Critique visuelle VLM (option)	confirme ou non les signaux ci-dessus sur une image croquis / rendu	ne corrige que si elle confirme un contrôle géométrique

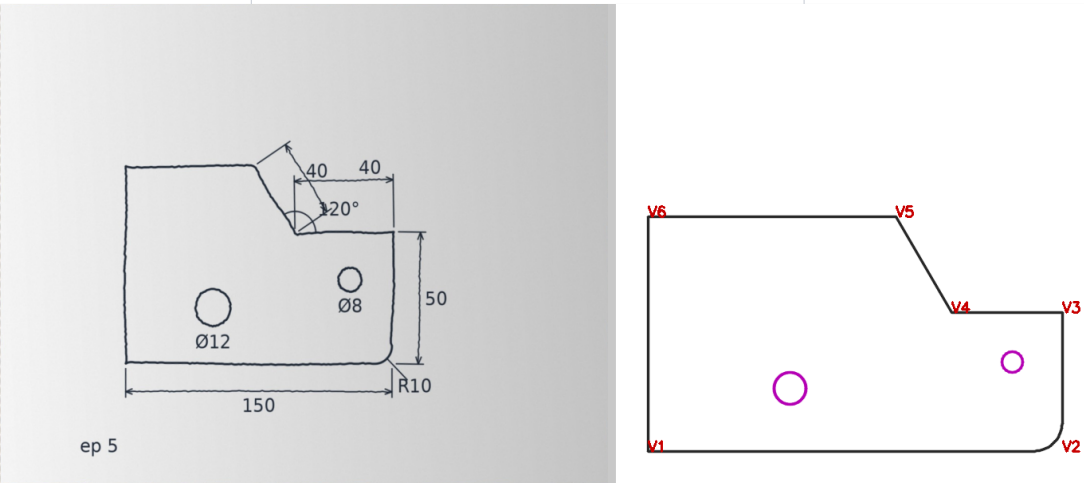


Figure 6 — Image de comparaison : croquis (gauche) et profil résolu (droite), après correction automatique de l'angle de 120° en 240° par l'agent critique.

9. Agent 4 — Synthèse « Code-as-CAD »

L'agent 4 écrit un script Python autonome `part.py`, puis l'exécute dans un sous-processus isolé (délai de 180 s). Le script est le vrai livrable CAO : lisible, paramétrique (cotes en tête de fichier), et relançable seul. Un gabarit déterministe couvre chaque forme ; un LLM codeur local (Qwen-Coder) peut ensuite modifier le script pour une fonction hors gabarit, avec jusqu'à 3 essais nourris par la trace d'erreur.

Forme	Solide 3D (build123d / OpenCASCADE)	Mise en plan (ezdxf)
Pièce plate	profil (lignes + arcs de congé en 3 points) → face → perçages soustraits → extrusion	contour, perçages et axes, cotes alignées, angulaires, radiales, diamétrales, épaisseur
Cylindre	Cylinder(r, h)	vue de face (Ø, hauteur, axe) + vue de dessus
Corps de révolution	demi-profil dans le plan XZ (lignes + arc du dôme) → révolution 360° autour de Z	silhouette complète et son axe, cotes du profil, R du dôme, Ø des faces
Sphère / disque	Sphere(r) / cylindre de faible épaisseur	cercle, axes, SØ ou Ø

- calques DXF : CONTOUR (0,5 mm), PERCAGES (0,5 mm), COTATION (0,18 mm), AXES (0,13 mm) ;
- congés et dômes en vrais arcs (pas de polyligne approchée) ;
- SVG rendu depuis le même DXF sur une page A4 paysage, épaisseurs de trait absolues ;
- STL avec tolérance proportionnelle à la taille (0,01 mm minimum) ; contrôle après exécution : validité du solide, volume, encombrement.

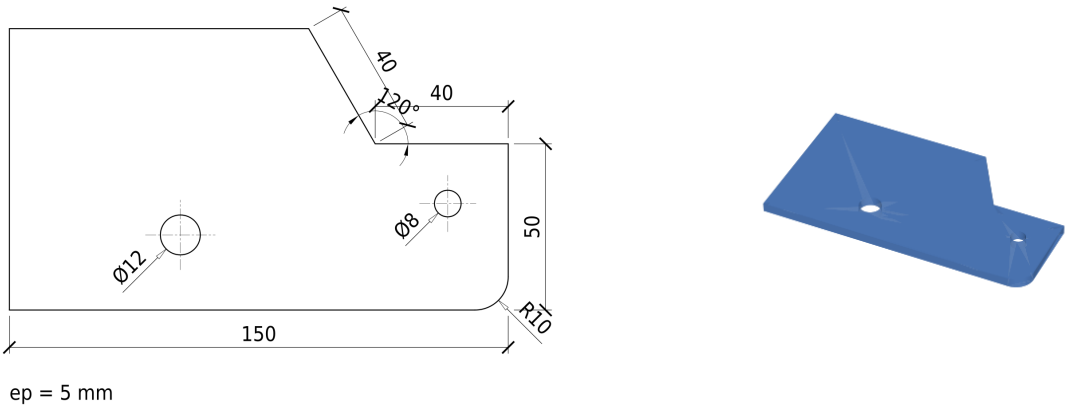


Figure 7 — Pièce plate de test : plan coté généré et solide 3D (150 × 84,641 × 5 mm, congé R10, perçages Ø12 et Ø8).

Extrait d'un part.py généré (corps de révolution)

```
PROFILE = [(0.0, 0.0), (250.0, 0.0), (500.0, 91.0), (500.0, 1091.0), (0.0, 1591.0)]
DOME = 3        # arête 3 : arc du dôme, centre sur l'axe

def build_solid():
    with BuildPart() as part:
        with BuildSketch(Plane.XZ):
            with BuildLine():
                for s in segments():
                    Line(s[1], s[2]) if s[0] == "line" else ThreePointArc(s[1], s[5], s[2])
            make_face()
        revolve(axis=Axis.Z)
    return part.part
```

10. Interface web de test

Une interface web locale (Flask, servie uniquement sur 127.0.0.1:8765) permet de tester le système sans ligne de commande. La visionneuse 3D (three.js) est embarquée dans le projet : aucune ressource n'est chargée depuis Internet. Les calculs sont exécutés un par un en arrière-plan ; la page interroge leur état toutes les 1,5 s.

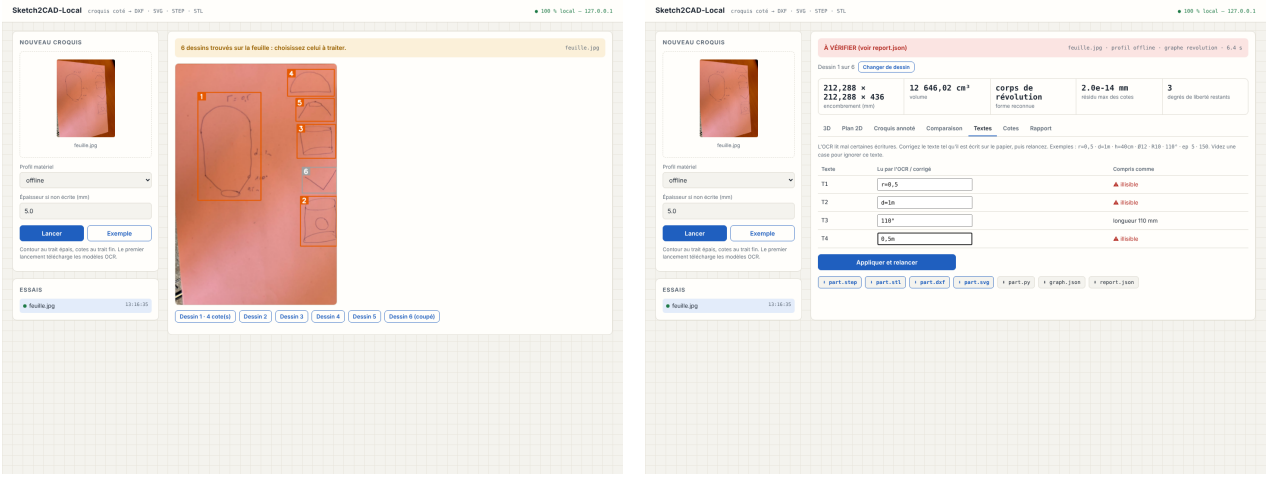


Figure 8 — Choix du dessin sur une feuille de 6 dessins, puis onglet Textes : correction des cotes mal lues par l'OCR avant relance.

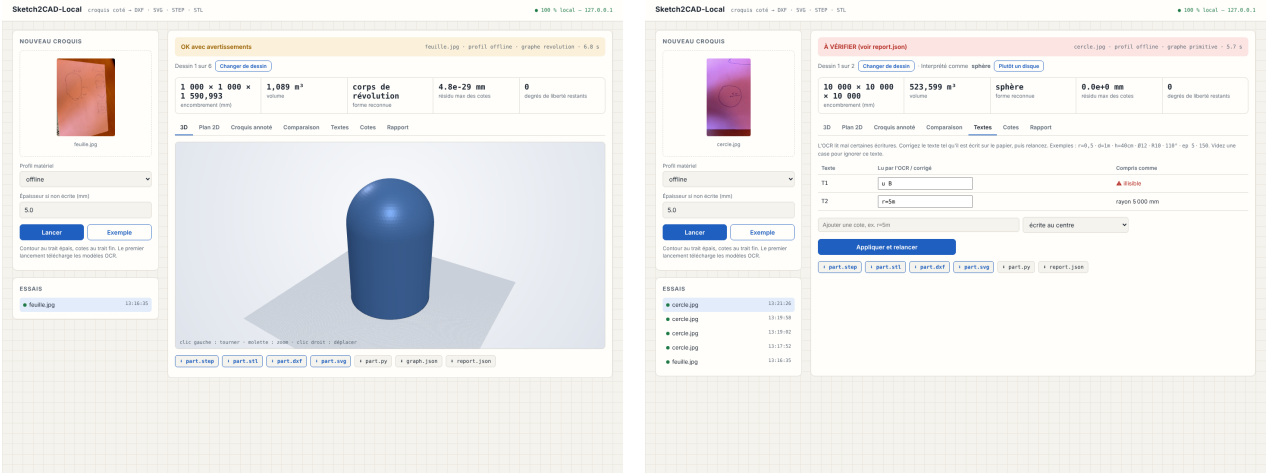


Figure 9 — Résultats dans l'interface : gélule après correction des textes (1,089 m³), et cercle interprété comme sphère après ajout de la cote « r=5m ».

Fonction	Détail
Nouvel essai	dépôt d'une photo ou exemple fourni ; profil offline / cpu / gpu4 / gpu8 ; épaisseur par défaut
Choix	dessin à traiter (feuille à plusieurs dessins), sphère ou disque (cercle seul)
Onglets	3D, Plan 2D, Croquis annoté, Comparaison, Textes, Cotes, Rapport
Textes	corriger un texte OCR, le vider pour l'ignorer, ajouter une cote manquante avec sa position (centre, droite, haut, bas, gauche) ; relance sans refaire l'OCR
Cotes	modifier une valeur ou l'élément coté, supprimer une cote ; recalcul en quelques secondes
Historique	essais précédents conservés dans sortie_web\, rouvrables d'un clic
Téléchargements	.step, .stl, .dxf, .svg, part.py, graph.json, report.json

API locale

Requête	Rôle
POST /api/jobs	crée un essai (photo + profil) et lance le pipeline
GET /api/jobs, GET /api/jobs/<id>	liste des essais ; état, rapport, dimensions, fichiers
GET /api/jobs/<id>/texts	textes lus sur le dessin traité
POST /api/jobs/<id>/rerun	relance avec un autre dessin, une interprétation, des textes corrigés ou ajoutés
GET /api/jobs/<id>/graph, POST .../resolve	cotes modifiables ; recalcul solveur + CAO
GET /files/<id>/<fichier>	images et livrables (?dl=1 pour télécharger)

11. Résultats et validation

11.1 Croquis traités

Croquis	Lecture / interprétation	Résultat	Statut
Plaque de test photographiée en biais (synthétique, hors échelle)	6 sommets, congé R10, perçages Ø12 et Ø8, angle 120° corrigé en 240° par la critique	150 × 84,641 × 5 mm ; écart à la vérité terrain $1{,}4 \cdot 10^{-14}$ mm	OK
Votre photo de cylindre	« F=0.5 » (r) et « d=2m » lus ; r sans unité pris en m ; d=2m à côté des génératrices = hauteur	Ø 1000 × 2000 mm ; 1,571 m³	OK avec avertissements
Votre feuille, dessin 1 (gélule)	3 textes illisibles + « 110 » sans ° ; corrigés en r=0,5 · d=1m · 110° · 0,5m	Ø 1000 × 1590,993 mm ; 1,089 m³ (vérifié : hémisphère 0,2618 + cylindre 0,7854 + tronc de cône 0,0417 m³)	OK avec avertissements (angle)
Feuille, dessins 2 et 3	cylindres reconnus (perçage latéral du n° 2 ignoré)	solides générés ; aucune cote écrite	À VÉRIFIER
Feuille, dessins 4 et 5	hémisphère et tronc de cône (base ouverte refermée par enveloppe convexe)	solides de révolution ; aucune cote écrite	À VÉRIFIER
Feuille, dessin 6 (V)	dessin ouvert	pas de solide, message explicatif	À VÉRIFIER
Votre photo de cercle	« 5 m » illisible (lu « u B ») ; choix sphère ; cote r=5m ajoutée	sphère Ø 10 m ; 523,6 m³	À VÉRIFIER tant que « u B » n'est pas vidé

Pour la gélule, l'angle « 110° » est pris comme angle intérieur à l'épaulement : le cône ne mesure alors que 91 mm de haut, alors que le dessin montre environ 143°. Le rapport le signale, car le sens de mesure voulu par le dessinateur ne peut pas être deviné de façon sûre.

11.2 Tests automatiques (34, tous réussis)

Groupe	Ce qui est vérifié
Analyse des cotes (17)	150, 80,5, 120°, R10, Ø12, ⌀ 8, ep 5, e=3, 012, l5O, F=0.5, d=2m, h = 40 cm, r=12, Ø12mm ; textes non cotes rejetés ; propagation d'unité ; re-analyse après correction
Solveur (5)	rectangle hors échelle exact, détection du sous-cotage, relâche d'une contrainte implicite, conflit impossible signalé, triangle par angles
Géométrie (1)	points de tangence et centre d'un congé
Bout en bout (2)	croquis synthétique hors échelle contre vérité terrain, avec correction de l'angle ; chemin VLM simulé (valeur corrigée par le modèle, cible invalide remplacée par l'heuristique)
Formes (6)	cylindre détecté et construit ; rectangle non pris pour un cylindre ; gélule résolue exactement ; cercle → question puis sphère ; dessin ouvert signalé ; feuille de 3 dessins (perçage rattaché)
Interface (1)	page, API, visionneuse 3D embarquée, refus d'un envoi sans image
Divers (2)	garde-fous de détection et d'analyse

11.3 Temps de calcul (environnement de test, CPU)

- premier passage d'un croquis : environ 30 à 110 s, essentiellement le chargement et l'exécution de PaddleOCR sur CPU ;
- relance après correction de textes ou de cotes : 5 à 7 s (pas d'OCR) ;
- solveur et génération CAO : quelques secondes au plus.
- Ces temps sont à mesurer sur votre machine.

12. Matériel, IA locale et utilisation du GPU

État actuel : le GPU n'est pas utilisé. Le profil par défaut est `offline` : aucun modèle de langage n'est appelé ; PaddleOCR est installé en version CPU ; le solveur et la CAO n'ont pas besoin de GPU. Ollama est installé et joignable sur votre PC, mais les modèles ne sont pas encore téléchargés.

Profil	Matériel	VLM (lecture / rattachement)	Codeur	Critique VLM
offline	n'importe quel PC	aucun : OCR + heuristiques	gabarit seul	non
cpu	CPU, ≥ 16 Go de RAM	modèle 2–3B sur CPU (lent)	gabarit seul	non
gpu4	GPU 4–8 Go	Qwen3-VL 2B ou Qwen2.5-VL 3B	Qwen2.5-Coder 3B	oui
gpu8	GPU 8–12 Go	**Qwen3-VL 8B** (≈ 6 Go en Q4) recommandé ; Qwen2.5-VL 7B en alternative	Qwen2.5-Coder 7B	oui

Avec moins de 12 Go, les modèles sont chargés l'un après l'autre (Ollama, `keep_alive: 0`). Ollama utilise automatiquement une carte NVIDIA ; la commande `ollama ps` indique si le modèle tourne sur le GPU. Les comparatifs publiés mesurent la lecture de documents imprimés, pas l'écriture manuscrite : le gain réel devra être mesuré sur vos croquis.

Prochaine étape recommandée

Faire relire les cotes par le VLM pour toutes les formes, PaddleOCR restant en secours : le modèle propose les textes, qui arrivent dans l'onglet Textes où l'utilisateur garde la main. C'est l'endroit où le GPU apporte le plus, car l'erreur dominante observée sur vos photos est la lecture de l'écriture, pas la géométrie.

13. Installation et utilisation

Étape	Action
1	Installer Python 3.10 à 3.12 (votre PC : 3.10.0, compatible).
2	Double-cliquer install.bat : environnement .venv, dépendances, diagnostic (python -m sketch2cad doctor).
3	Option IA : installer Ollama puis « ollama pull qwen3-vl:8b » ; choisir le profil gpu8 dans l'interface.
4	Double-cliquer web.bat : le navigateur s'ouvre sur http://127.0.0.1:8765.
5	Déposer la photo, choisir le dessin si demandé, corriger les textes si besoin, télécharger les fichiers.

```
python -m sketch2cad web                                  # interface de test
python -m sketch2cad run croquis.jpg -o sortie\piece1     # pipeline complet
python -m sketch2cad run feuille.jpg --drawing 2           # 2e dessin de la feuille
python -m sketch2cad run cercle.jpg --interpret sphere     # cercle seul -> sphère
python -m sketch2cad solve sortie\piece1\graph.json       # rejouer après correction
python -m sketch2cad --profile gpu8 doctor                # vérifier modèles et GPU
```

Bien dessiner pour la machine

- contour au trait épais, cotation au trait fin ;
- une cote par élément, près de sa ligne de cote ; R pour les congés, Ø pour les diamètres, ep pour l'épaisseur ; préférer « h = » à « d = » pour une hauteur ;
- pour un solide : dessiner l'ellipse des bases (indice 3D) ; fermer les contours ;
- feuille entière dans la photo, éclairage homogène, sans ombre sur le dessin.

14. Organisation du code

Fichier	Lignes	Contenu
sketch2cad/perception.py	429	agent 1 : image, contour, perçages, routeur de formes, Set-of-Mark
sketch2cad/ocr.py	304	OCR (Paddle, EasyOCR, Tesseract), analyse des cotes, unités, re-analyse
sketch2cad/sheet.py	100	découpage d'une feuille en dessins
sketch2cad/primitives.py	206	détection et cotes du cylindre
sketch2cad/revolution.py	352	corps de révolution : silhouette, axe, ellipses, dôme, demi-profil, graphe
sketch2cad/graph_builder.py	288	agent 2 : heuristique hongroise, VLM, graph.json
sketch2cad/llm.py	67	client Ollama local (sans dépendance)
sketch2cad/solver.py	279	agent 3 : moindres carrés, relâche, DDL, conflits, congés
sketch2cad/critic.py	166	agent 5 : contrôles et corrections, image de comparaison
sketch2cad/cad.py, profile.py	733	agent 4 : gabarits part.py (plaque, cylindre, révolution, sphère)
sketch2cad/pipeline.py	370	orchestration, profils, statut et rapport
sketch2cad/webapp.py + page web	781	serveur Flask local, interface, visionneuse 3D
sketch2cad/cli.py, schema.py	241	commandes, structures de données (Pydantic)
tests/, tools/	476	34 tests, générateur de croquis hors échelle avec vérité terrain

Dépendances principales : OpenCV, NumPy, SciPy, Shapely, Pydantic, PyYAML, ezdxf, build123d (OpenCASCADE), PaddleOCR/PaddlePaddle, Flask ; optionnelles : Ollama et modèles Qwen, Tesseract, EasyOCR.

15. Limites et perspectives

Limites actuelles

- **Lecture de l'écriture manuscrite** : principale source d'erreur sur des photos réelles ; compensée par l'onglet Textes, à améliorer par la lecture VLM.
- **Pièces combinées** non traitées : perçage dans un cylindre, assemblages, poches, chanfreins, tôlerie pliée, vues multiples.
- **Ambiguïtés de notation** : « d = » (diamètre ou hauteur) et sens de mesure d'un angle sont tranchés par la position, avec avertissement.
- **Position des perçages** non cotée : prise sur le dessin (signalée).
- **Validation** : 3 photos réelles seulement ; aucun taux de réussite statistique encore établi.

Feuille de route

Priorité	Travail	Gain attendu
1	Lecture des cotes par Qwen3-VL sur GPU pour toutes les formes	moins de corrections manuelles
2	Jeu de 30 à 50 photos réelles avec vérité terrain	mesurer le taux de réussite et régler les seuils
3	Perçages et éléments sur les solides de révolution	cylindre percé, bouteilles à goulot
4	Poches, chanfreins, pièces combinées	couvrir davantage de pièces d'atelier
5	Tôlerie pliée et vues multiples	pièces pliées, mises en plan multi-vues

Conclusion

Sketch2CAD-Local montre qu'une séparation stricte entre perception et calcul permet d'obtenir, à partir d'un croquis hors échelle, des géométries exactes au millième de millimètre et des fichiers CAO directement exploitables, sur un poste ordinaire et sans quitter la machine. Sur vos photos, les formes sont désormais reconnues ; la qualité finale dépend surtout de la lecture des cotes, que l'utilisateur peut contrôler et corriger, et que le passage au VLM sur GPU doit fiabiliser.